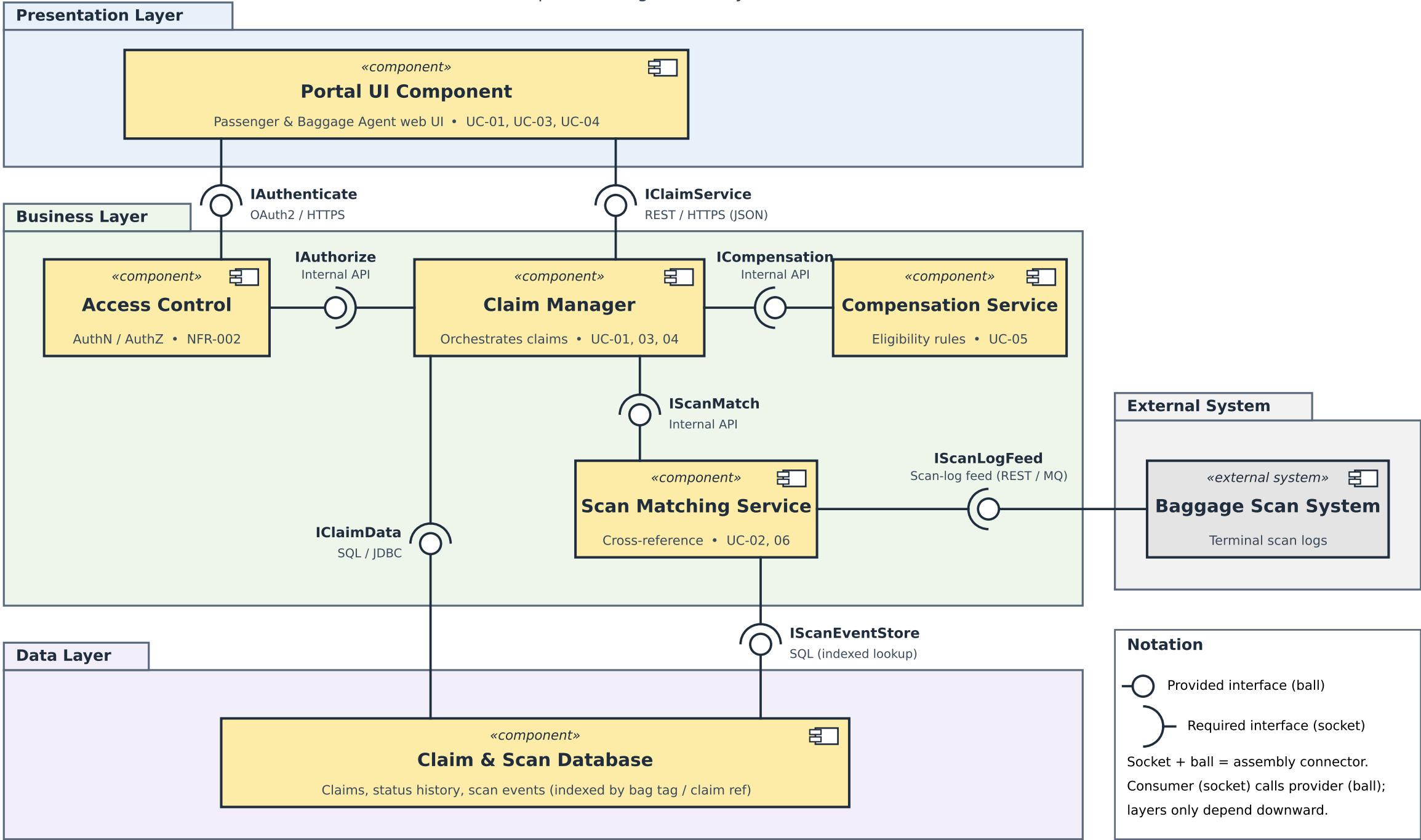


Airport Lost Luggage Claim & Tracking Portal

UML Component Diagram — Layered Architecture (Lab 3)



Interface catalogue and data flow (Provider → Consumer)

IAuthenticate	Access Control → Portal UI credentials → session token
IClaimService	Claim Manager → Portal UI claim form / claim ref, status, timeline
IAuthorize	Access Control → Claim Manager token + role → allow / deny
ICompensation	Compensation Svc → Claim Manager claim data → eligibility, workflow started

IScanMatch	Scan Matching → Claim Manager bag tag → match result, recovery alert
IScanLogFeed	Baggage Scan System → Scan Matching unrouted scan logs, multi-leg events
IClaimData	Database → Claim Manager create / read / update claims and status
IScanEventStore	Database → Scan Matching store / query scan timeline (<200 ms)

Lab 3 – Architectural Justification

Airport Lost Luggage Claim & Tracking Portal | Component Modelling & Architectural Pattern Selection

Architecture Selection

We chose **Layered Architecture for the Airport Lost Luggage Claim & Tracking Portal**. The system is organised into a Presentation layer (Portal UI), a Business layer (Access Control, Claim Manager, Scan Matching Service, Compensation Service) and a Data layer (Claim & Scan Database), with the Baggage Scan System as an external source. Dependencies only run downward through defined interfaces.

Style	Fit for this scenario	Verdict
Layered	Maps directly to the claim workflow; strong consistency; single deployable unit; simple to secure and test.	Selected
Microservices	Independent scaling, but claim, scan-match and compensation data must stay consistent, and extra network hops threaten the 200 ms target. Too much operational overhead for 5 FRs and 2 NFRs.	Rejected
Client-Server	Simple, but a single central server gives no internal structure and is a single point of failure for a portal used across many terminals.	Rejected

Two Reasons for the Choice

1. The layers mirror the use cases and keep the design simple. Passenger and agent interactions (UC-01, UC-03, UC-04) belong in the Presentation layer; claim handling, cross-referencing scan logs (UC-02, FR-001), recovery alerts (UC-06) and compensation (UC-05, FR-005) belong in the Business layer; claims and scan events belong in the Data layer. This one-domain workflow is small and well defined, so a layered design is quicker to build, test and maintain than a distributed one.

2. Business rules and integrations are isolated and replaceable. The airline-configured compensation conditions live only inside the Compensation Service (ICompensation), and the connection to terminal scan systems lives only behind IScanLogFeed. Either can change, or be swapped for another component offering the same interface, without touching the UI or database. A single database also keeps claim status, matches and compensation state consistent.

Security Advantage (NFR-002)

Every request enters through one path. The Portal UI authenticates via IAuthenticate, and the Claim Manager calls IAuthorize before returning or changing any claim, so passengers see only their own claims and only authorised baggage service agents can update status. The UI has no direct data access; only the Business layer reaches the database (IClaimData, IScanEventStore). This gives one enforcement point, a small attack surface, and a clear place to apply TLS and audit logging.

Performance Benefit (NFR-001)

The tracking timeline request travels through in-process calls inside one deployable unit, avoiding the network hops of a microservice chain. Scan events are kept in an indexed store (IScanEventStore, keyed by bag tag / claim reference), so the complete multi-leg history is a single indexed query, which supports the under-200 ms target. Scan-log matching is done when a claim is submitted, not on every timeline read. The main trade-off is that layers scale together; this is handled by running several copies of the stateless Business layer behind a load balancer.